



Universidade de Brasília

Departamento de Ciência da Computação

Arquitetura RISC-V



Instruction Set Architecture - ISA

- A ISA define os recursos que o programador pode utilizar para desenvolver programas em determinado processador
- Informações típicas:
 - Quais instruções o processador executa
 - Qual o formato destas instruções
 - Que tipos de dados são suportados
 - Quais os registradores estão disponíveis
 - Que modos de endereçamento são utilizados



RISC x CISC

- Historicamente, as arquiteturas dos computadores evoluíram incorporando instruções cada vez mais complexas
- CISC (Complex Instruction Set Computer)
 - Grande variedade de instruções
 - uma instrução que realiza um procedimento complexo reduz o número de acessos à memória
 - reduzir a distância semântica (*semantic gap*) para linguagens de alto nível
 - Entretanto:
 - Controle do processador complexo
 - Pouco uso de instruções complexas (20% ISA usado em 80% dos casos)



RISC x CISC

- RISC (Reduced Instruction Set Computer - anos 80)
 - Implementação de um conjunto reduzido de instruções
 - Redução da complexidade do hardware de controle do processor, o que implica:
 - Maior velocidade na execução de instruções simples
 - Instruções de mesmo tamanho
 - Instruções específicas para acesso à memória
 - Exemplos:
 - MIPS, RISC-V, ARM



Arquitetura do RISC-V

- Desenvolvida na UC Berkeley como uma ISA aberta
 - Projeto iniciado em 2010 por alunos do Patterson
- Gerida pela Fundação RISC-V (riscv.org)
- Ampla gama de aplicações
 - Desde sistemas embarcados até supercomputadores
- Vários subconjuntos de instruções
 - RV32I - conjunto básico para instruções com inteiros de 32 bits
 - RV32E - voltado a sistemas embarcados
 - RV64I - conjunto básico para instruções com inteiros de 64 bits
 - RV128I - conjunto básico para instruções com inteiros de 128 bits



Arquiteturas RISC-V

RV32 – registradores de 32 bits

RV64 – registradores de 64 bits

RV128 – registradores de 128 bits

Tipo de instruções	Sufixo
ISA de inteiros	I
Instruções de Multiplicação e Divisão	M
Instruções atômicas (sincronização de memória)	A
Instruções de ponto flutuante (precisão simples)	F
Instruções de ponto flutuante (precisão dupla)	D
ISA Geral	IMAFD = G
Conjunto reduzido para sistemas embarcados	E

Modo normal: Instruções com 32 bits de tamanho

Modo condensado: Instruções com 16 bits de tamanho

Modo expandido: Instruções com $n \times 16$ bits de tamanho (48,64,96,...)



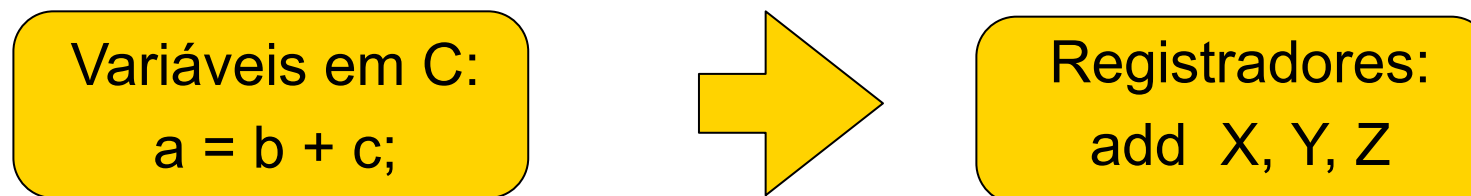
Traduzindo de C para Linguagem de Montagem

■ Operações:

- o *assembly* deve permitir realizar todas as operações definidas em uma linguagem de alto nível. Usualmente temos uma instrução em *assembly* para cada operação:

- + : add
- - : sub
- & : and
- etc

- ***Variáveis*** em um programa são associadas a ***registradores*** no processador:





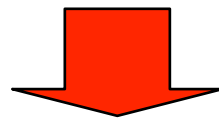
Operações do Hardware

- Todo computador deve ser capaz de realizar operações aritméticas.

ex...: *add a,b,c* $\longleftrightarrow$ $a = b + c$

- Instruções aritméticas no RISC-V têm formato fixo, realizando somente uma operação e tendo três “variáveis”

ex: $a = b + c + d + e$



Comentários

add a, b, c	# a = b + c
add a, a, d	# a = b + c + d
add a, a, e	# a = b + c + d + e

Somente uma
instrução por
linha



Operações do Hardware

- Exigir que toda instrução tenha exatamente três operandos condiz com a filosofia de manter o hardware simples: hardware para número variável de parâmetros é mais complexo que para número fixo.

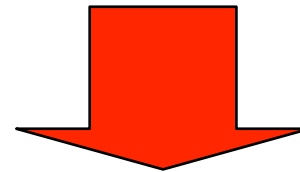
Princípio #1 para projetos: Simplicidade
favorece a regularidade



Exemplo 1

- Qual o código gerado por um compilador C para o seguinte trecho?

```
a = b + c;  
d = a - e;
```



```
add a, b, c  
sub d, a, e
```

```
# a = b + c  
# d = a - e
```



Exemplo 2

- Qual o código gerado por um compilador C para o seguinte trecho?

$f = (g + h) - (i + j);$

Somente uma operação é feita por instrução: necessidade de variáveis temporárias.

add t0, g, h

add t1, i, j

sub f, t0, t1

temporário t0 = g + h

temporário t1 = i + j

#f = (g + h) - (i + j)



Operandos e Registradores

- Registradores do RISC-V RV32I são de 32 bits
- Blocos de 32 bits são chamados de palavra
- Número de registradores é limitado: RISC-V $\rightarrow$ 32 registradores, numerados de 0 a 31
 - acesso mais rápido, interno ao *chip*
 - fácil acesso
- **Princípio #2 para projetos: menor é mais rápido**
 - Um número muito grande de registradores aumentaria o período de clock.



Operandos e Registradores

- No RISC-V existe uma convenção para nomear registradores na forma x_i :
 - $x_0, x_1, x_2, \dots, x_{30}, x_{31}$
- Variáveis em um programa C são usualmente associadas a endereços de memória
 - Para executar operações no processador, os dados tem que ser transferidos para registradores
 - As operações lógico - aritméticas em um processador RISC são realizadas sobre registradores
 - Assim, existe um mapeamento de variáveis do C para registradores do processador



Registadores RV

Registrador	Nome	Descrição	Preservado?
x0	zero	Constante zero <i>hardwired</i>	-
x1	ra	Endereço de retorno	Sim
x2	sp	Stack Pointer	Sim
x3	gp	Global Pointer	-
x4	tp	Thread Pointer	-
x5	t0	Temporário	Não
x6-7	t1-2	Temporários	Não
x8	s0/fp	Saved reg / frame pointer	Sim
x9	s1	Saved Register	Sim
x10-11	a0-1	Argumentos / val retorno	Não
x12-17	a2-7	Argumentos função	Não
x18-27	s2-11	Saved Registers	Sim
x28-31	t3-6	Temporários	Não



Exemplo 2...

- Considerando a convenção adotada, podemos associar:

□ $f \Rightarrow x19$

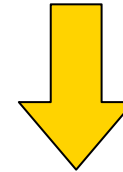
□ $g \Rightarrow x20$

□ $h \Rightarrow x21$

□ $i \Rightarrow x22$

□ $j \Rightarrow x23$

$$f = (g + h) - (i + j)$$



add **x5, x20, x21**

add **x6, x22, x23**

sub **x19, x5, x6**

temporário x5 = g + h

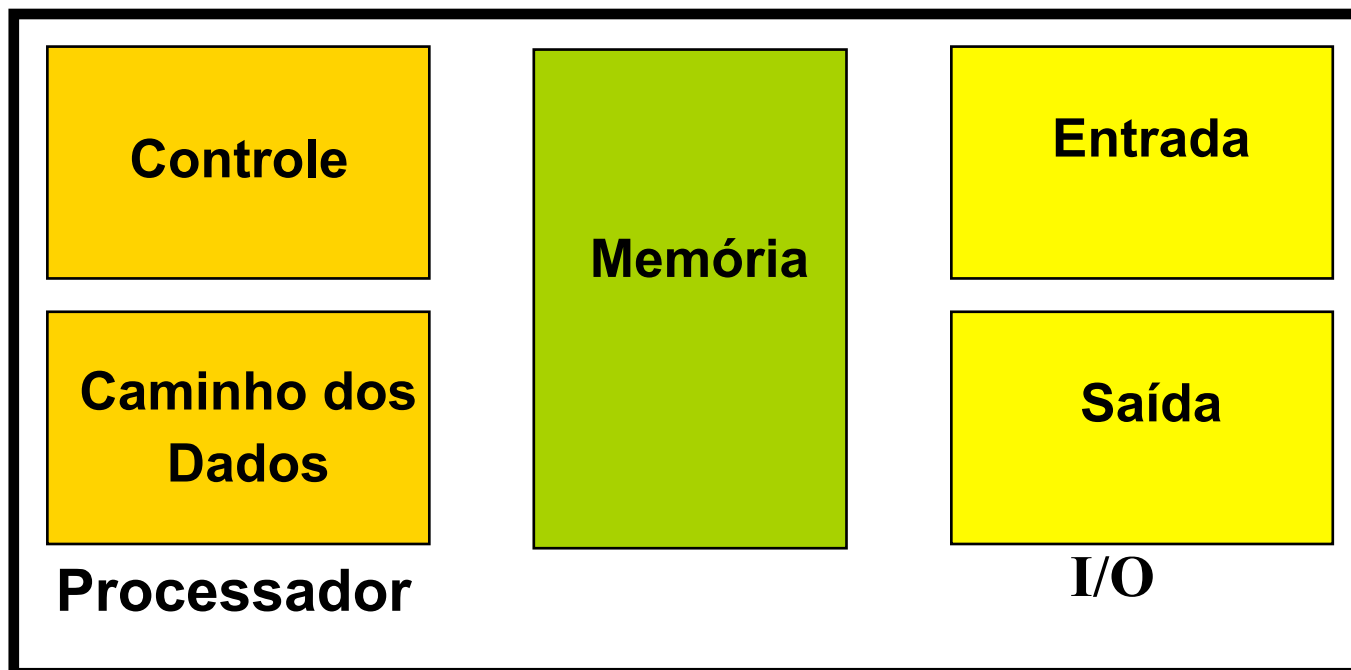
temporário x6 = i + j

f = (g + h) - (i + j)



Registradores vs Memória

- Operandos de instruções aritméticas devem ser registradores (32 registradores disponíveis).
- Compilador associa variáveis a registradores.
- E programas com várias variáveis ?
 - São mantidas na memória





Organização da Memória

- Vista como um grande *array* unidimensional, com endereços sequenciais, começando em 0
- Um **endereço** de memória é um **índice** no *array*.
- "*Byte addressing*" significa que o índice aponta para um *byte* na memória.



Processador



Barramento

⋮	⋮
123	3
77	2
101	1
21	0

Dados Endereços

Memória

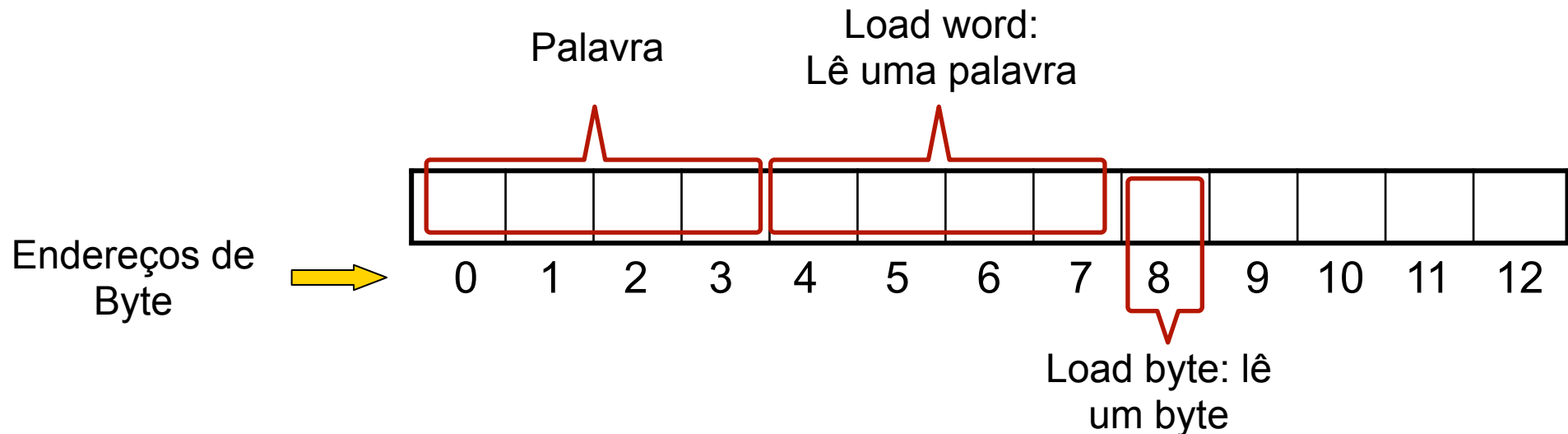




Organização da Memória

- As palavras de 32 bits são divididas em 4 bytes
- O RISC-V pode endereçar um byte ou uma palavra inteira

Instrução define se é
byte ou *word*



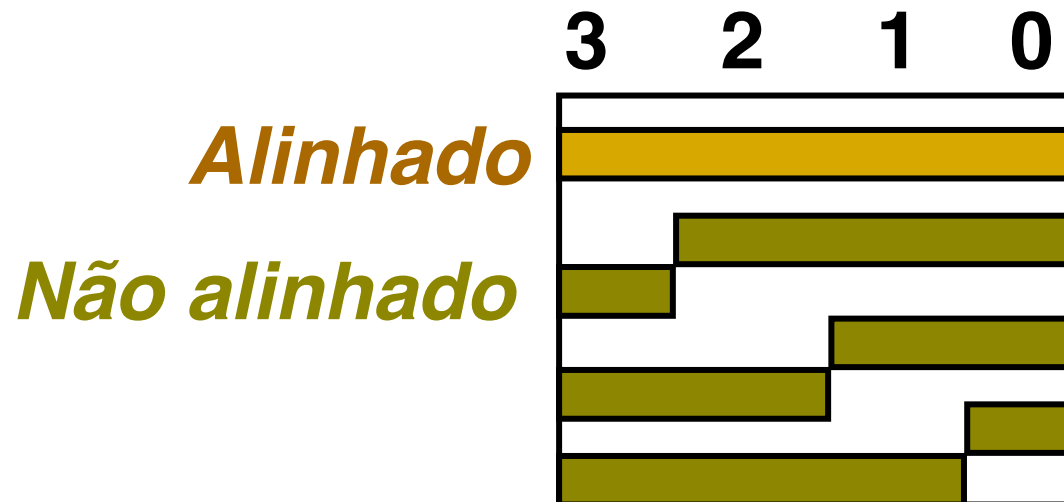


Organização da Memória

2^{32} *bytes* com endereços de *byte* de 0, 1, 2, 3, ... $2^{32}-1$

2^{30} *words* com endereços de *byte* de 0, 4, 8, ... $2^{32}-4$

Words são **alinhadas**, i.e., quais são os valores dos 2 bits menos significativos do endereço de uma *word*?

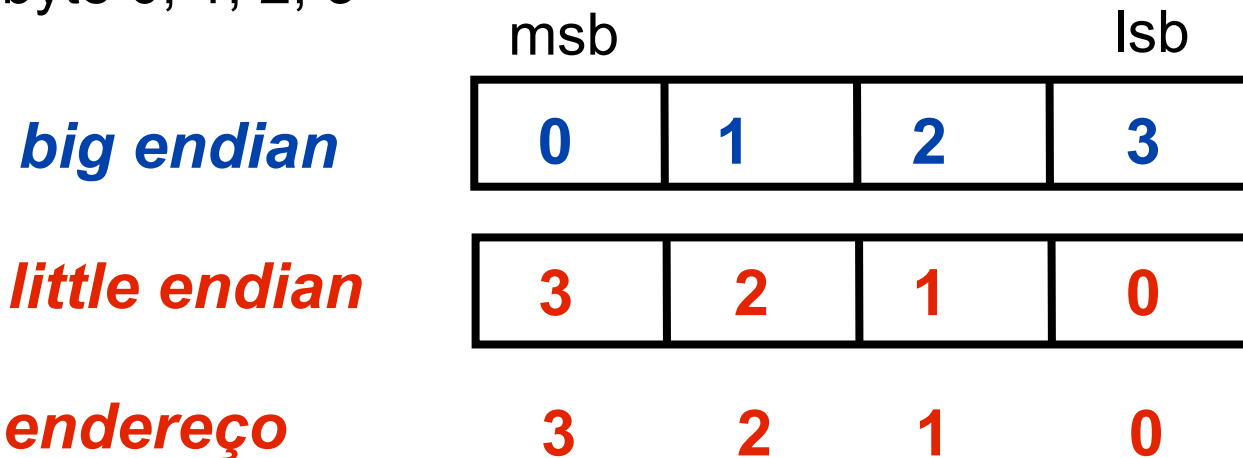




Ordenamento dos *Bytes*

- Processadores podem numerar bytes dentro de uma palavra, de tal forma que o byte com o menor número é o mais a esquerda ou o mais a direita. Isto é chamado de byte order.

□ Ex: .byte 0, 1, 2, 3



- Big endian: IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA
- Little Endian: Intel 80x86, MIPS, DEC Vax, DEC Alph



Transferindo dados da memória

- A instrução de transferência de dados da memória para o registrador é chamada de load.

No RISC-V, o nome da instrução é: **lw** (*load word*)

- Formato:

lw *registrador destino, constante (registrador base)*

- Ex:

g = h + *a;

a => x20, g => x21, h => x22

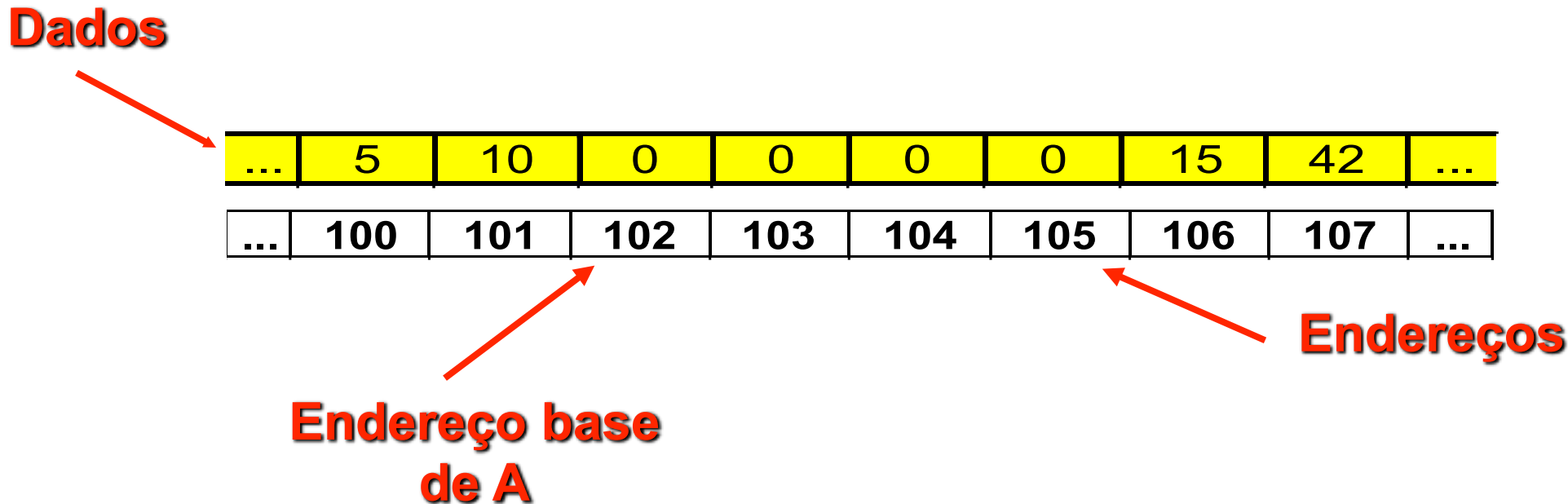
```
lw    x5, 0(x20)
add   x21, x22, x5
```

```
# temporário x5 = *a
# g = h + *a
```



Vetor de Bytes na Memória

Dados



...	5	10	0	0	0	0	15	42	...
-----	---	----	---	---	---	---	----	----	-----

...	100	101	102	103	104	105	106	107	...
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

**Endereço base
de A**

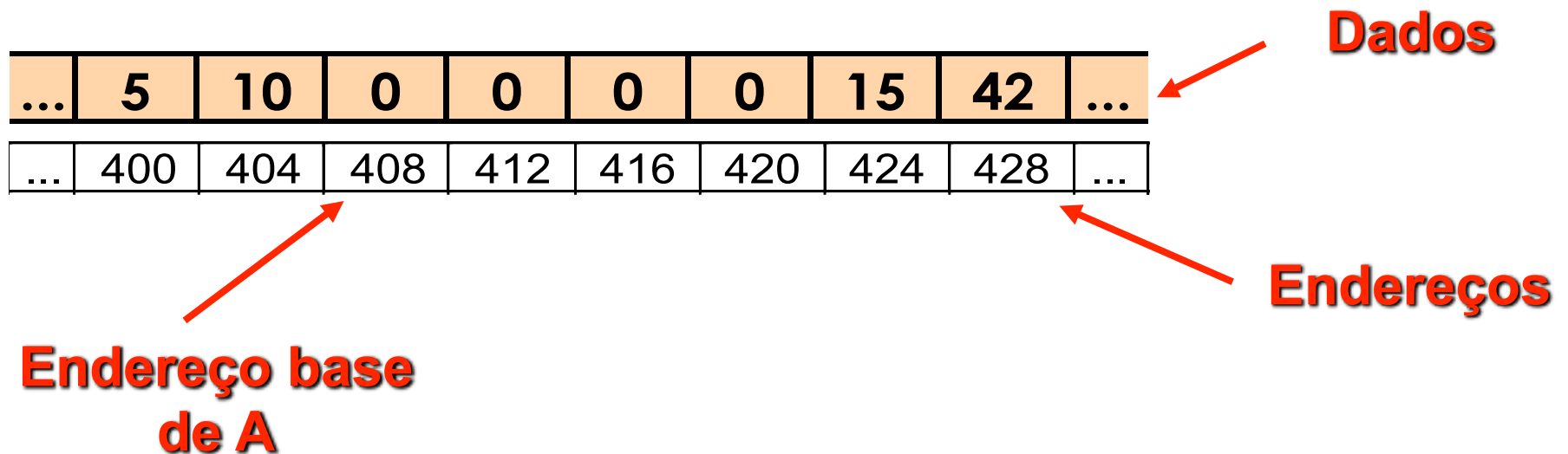
Endereços

- Vetor de bytes $A = [0, 0, 0, 0, 15]$, com 5 posições, começando no endereço de memória 102. Este endereço é chamado de endereço base do vetor. Assim, 102 é o endereço de $A[0]$, 103 o de $A[1]$, ..., 106 o de $A[4]$.



Vetor de *words*

- Cada posição do vetor (de inteiros) é uma palavra, e portanto ocupa 4 bytes



- Vetor A = [0,0,0,0,15], com 5 posições, começando no endereço de memória 408. Assim, 408 é o endereço de A[0], 412 o de A[1], 416 o de A[2], 420 o de A[3] e 424 o de A[4].



Exemplo

- Suponha que o vetor A tenha 100 posições, e que o compilador associou a variável h ao registrador x21. Temos ainda que o endereço base do vetor A é dado em x22. Qual o código para:

$A[12] = h + A[8]$?

- A nona posição do vetor A, $A[8]$, está no offset $8 \times 4 = 32$

```
lw x9, 32(x22)      # temporário x9 = A[8]
add x9, x21, x9      # temporário x9 = h + A[8]
```

- A décima-terceira posição do vetor A, $A[12]$, está no offset $12 \times 4 = 48$

```
sw x9, 48(x22)      # armazena x9 em A[12]
```




Transferindo dados para a memória

- A instrução de transferência de dados de um registrador para a memória é chamada de store.

No RISC-V, o nome da instrução é:
sw (store word)

- Formato:

sw registrador fonte, constante (registrador base)

- Endereço de memória acessado é dado pela soma da constante (*offset*) com o conteúdo do registrador base



Exercício

- Temos ainda que o endereço base do vetor A é dado em x22, e que as variáveis i e g são dadas em x20 e x21, respectivamente. Qual o código para

$$A[i+g] = g + A[i] - A[0] ?$$



Convenção do uso de Registradores

O registrador **x0** contém sempre o valor **0** (*hardwired*).

O registrador **x1 (ra)** armazena o endereço de retorno quando é executada a instrução de salto para função

O registrador **x2 (sp)** é usado como ponteiro para a pilha



Convenção do uso de Registradores

O registrador **x3 (gp)** é o *Global Pointer*, aponta para o segmento de dados globais estáticos

O registrador **x4 (tp)** é chamado *Thread Pointer*, ponteiro para área de dados local da *thread* em execução

Os registradores **x5-7 (t0-2)** e **x28-31 (t3-6)** são usados para armazenamento de dados temporários



Convenção do uso de Registradores

Os registradores **x8-9** e **x18-27** são usados para armazenamento de variáveis não temporárias

Os registradores **x10-17 (a0-7)** são usados para passagem de parâmetros para funções

Os registradores **x10 e x11** são também utilizados para retorno de valores de funções

O registrador **x8 (s0 - fp)** pode ser alternativamente utilizado como *Frame Pointer*, ponteiro para área local de dados na pilha